

National University of Computer and Emerging Sciences, Lahore Campus

	Course:	Parallel and Distributed Computing	Course Code:	CS3006
	Program:	BS(CS)-7G	Semester:	7th Semester
	Due Date	Friday, 11 September 2026	Total Marks:	100 marks
	Instructor		TA	
	Type:	Assignment 1		

Multi-Core and SIMD Parallelism

Estimated Time to Complete: 8–10 hours (spread across the week)

1. Objectives

By completing this assignment, you will:

- Parallelize a compute-bound image-rendering workload across multiple threads using static, contiguous work assignment, and reason about why naive decomposition leads to load imbalance.
- Write vectorized code by hand against an explicit SIMD instruction model, including correct use of lane masks for control flow, early termination, and edge cases where array size is not a multiple of vector width.
- Parallelize a memory-bandwidth-bound workload (AXPBY) and connect its scaling behavior to the compute-bound case, building intuition for when adding threads helps and when it does not.

2. Prerequisites & Setup

- A C/C++ toolchain (gcc/g++ or clang) on Linux/macOS, or WSL on Windows.
- Basic gnuplot / Python (matplotlib) / Excel for plotting speedup graphs(either is fine).

You have been provided a starter code archive containing three self-contained programs. Extract it and keep the folder structure as given: each program builds independently with its own Makefile.

```
ASSIGNMENT_1_PDC/  
|-- program 1/ (Part 1 - Fractal Rendering with Threads)  
|-- program 2/ (Part 2 - SIMD Vectorization)  
'-- program 3/ (Part 3 - AXPBY performance study)
```

Inside each program folder, run `make` to build, then run the produced executable with `-h` or `--help` to see available flags. Details for each program follow below.

3. The Tasks

Part 1: Parallel Fractal Rendering with Threads (20 marks)

You are given a serial reference program in **program 1/** that renders an escape-time fractal defined by $z_{n+1} = z_n^3 + c$ (a cubic variant, not the usual Mandelbrot $z^2 + c$). Each pixel maps to a point c in the complex plane; brighter pixels took more iterations to escape and cost more to compute, so the workload is compute-bound but not uniform across the image.

Files

- **fractalSerial.cpp** – serial reference implementation (correct, given, do not modify).
- **main.cpp** – test/timing harness (do not modify).
- **fractalThread.cpp** – you implement this. All your work goes inside **fractalWorker()**.

Fixed configuration (do not change)

900×601 image, `maxIterations = 300`, View 1: $x \in [-1.45, 0.85]$, $y \in [-1.05, 1.05]$; View 2: $x \in [-0.90, -0.30]$, $y \in [0.30, 0.90]$; max 16 threads.

Build & run

```
cd "program 1"
make
./fractal -t 2
```

- `-t N` threads for the run (default 2)
- `--view 1|2` which view of the fractal to render (default 1)
- `--sweep` run 1..8 threads and print a speedup table
- `--check` verify parallel output matches the serial reference

Work through these five stages, in order

1. Spatial decomposition (2 threads only): thread 0 computes the top half of the image, thread 1 the bottom half.
2. Generalize to 1-8 threads: contiguous row blocks, one per thread. Height (601) doesn't divide evenly, so don't drop or duplicate any row.
3. Per-thread timing: record each thread's own work time in `threadTimes[threadId]` (check it isn't `nullptr` first; some calls omit it).
4. Improve to roughly 7-8x speedup at 8 threads on both views, with no synchronization allowed and one general policy for all thread counts. Hint: change which rows a thread owns, not how many.
5. Run at 16 threads: no code change needed, just report what you observe.

Deliverable

Your completed **fractalThread.cpp** (naive and improved versions), a speedup-vs-threads graph for both views (from `./fractal --sweep --check`), per-thread timing data, and a written explanation (under a page) covering: is speedup linear? What does the 3-thread datapoint reveal on both views, and why? What did your per-thread timings show? What mapping

change did you make in stage 4, and what speedup did it get you? Did 16 threads help over 8, and why or why not?

Part 2: SIMD Vectorization – “Clamped Alternating Series” (20 marks)

In program 2/main.cpp you are given `clampedAltSerial()`, the serial reference. Rather than real AVX2 intrinsics, this part uses the course’s simulated vector instruction set defined in `PDCvector.h`, so you can inspect vector state directly while debugging.

Files

- `PDCvector.h` – the simulated vector instruction set. Read it before writing any code. You should not need to modify it, other than editing `#define VECTOR_WIDTH` to sweep widths.
- `main.cpp` – contains `clampedAltSerial()` (given); `absSerial()` / `absVector()`, a fully worked example (study it first; it has one deliberate gap, it does not handle array sizes that aren’t a multiple of `VECTOR_WIDTH`); `clampedAltVector()`, which you implement; and `dotProductSerial()` / `dotProductVector()`, optional bonus, where you implement `dotProductVector()`.

Build & run

```
cd "program 2"
make
./altseries -s 10000
```

- `-s N` array size to test with (default 10000)
- `-l` print a full instruction log
- `-b` also run the optional dot-product bonus test

The task

For every element i :

```
acc = values[i]
for step j = 0, 1, 2, ... while j < counts[i]:
    if j is even: acc = acc * values[i]
    else: acc = acc + values[i]
    if acc > caps[i]:
        acc = caps[i]
        break // this element is done, stop early
if acc < 0.0001: acc = 0
output[i] = acc
```

Every element has its own step budget (`counts[i]`) and its own cap (`caps[i]`), and an element can finish before using its full step budget if it clamps early. Your vectorized version must match the serial output exactly for any N and any `VECTOR_WIDTH`.

Deliverable

Your completed `clampedAltVector()`. Run with `VECTOR_WIDTH` set to 2, 4, 8, and 16 (edit `PDCvector.h`, rebuild each time) and record the reported Vector Utilization for each width. In your write-up (under half a page): does utilization increase, decrease, or stay the same as width grows? Explain why, in terms of how many lanes in a wide vector are likely still “clamped off” from earlier steps while other lanes in the same vector keep going.

Optional (5 bonus marks): your completed `dotProductVector()`, using `_pdc_hadd_float` / `_pdc_interleave_float` so the reduction costs fewer than $O(N)$ vector instructions.

Part 3: AXPBY – A Memory-Bandwidth-Bound Workload (20 marks)

AXPBY computes `result[i] = alpha * X[i] + beta * Y[i]` over a large array (program 5/prog5_axpby/), a two-coefficient variant of BLAS `saxpy`, with `alpha = 2.5`, `beta = -1.5` (fixed, do not change). Unlike the fractal renderer, it performs very little arithmetic per byte moved from memory, so its performance is limited by memory bandwidth rather than compute throughput.

Files

- `axpbySerial.cpp` – serial reference implementation (correct, given, do not modify).
- `main.cpp` – test/timing harness (do not modify).
- `axpbyThread.cpp` – you implement this. All your work goes inside `axpbyWorker()`.

Build & run

```
cd "program 5/prog5_axpby"
make
./axpby -t 4
```

- `-n N` array size (default 20000000)
- `-t threads` threads for the run (default 4)
- `--sweep` run 1..8 threads and print a table
- `--check` verify parallel output matches serial exactly

The task

Required decomposition: contiguous static blocks; thread t gets one unbroken range of indices (not interleaved, unlike Part 1). N won't always divide evenly by the thread count, so the remainder must be covered, not dropped. No mutex needed; each thread writes a disjoint range of `result`.

Deliverable

Your completed `axpbyThread.cpp`; a speedup graph from `./axpby --sweep --check` across 1–8 threads (time, GB/s, and speedup); results for at least three values of N , including one not divisible by 64 (e.g. `-n 1000003`), confirming `--check` passes for each; and a written comparison (under half a page) against your Part 1 fractal curve: which scales better, and why (arithmetic intensity)? The program moves `4*N*sizeof(float)` bytes even though each element only does 2 reads + 1 write; explain the extra factor.

4. Deliverables Checklist

- Source code for Part 1 (`fractalThread.cpp`, naive and improved versions, configurable thread count).
- Source code for Part 2 (`clampedAltVector()`, and `dotProductVector()` if attempting the bonus).
- Source code for Part 3 (`axpbyThread.cpp`, configurable thread count).
- Two speedup graphs (Fractal, AXPBY) generated from your own measured timings, for all required thread counts and, for Part 1, both views.

- Vector-utilization results for `VECTOR_WIDTH = 2, 4, 8, and 16`.
- One combined PDF report containing: both speedup graphs, per-thread timing data and the 3-thread explanation, the vector-utilization results and explanation, and the Part 3 compute-vs-memory-bound comparison.
- All files zipped as `RollNumber_Assignment1.zip`

5. Submission Instructions

Submit via Google Classroom (GCR) before the deadline. Late submissions are not allowed. Name your zip file exactly as specified above; submissions that don't follow the naming convention may be penalized.

- Every student must be able to correctly justify the code, their measured numbers, and their reasoning during viva. If a student is unable to correctly justify their own solution during evaluation, that assignment will also be marked a straight **0**, regardless of the code's correctness.

6. Grading Breakdown

Section	Marks	What earns it
Part 1 – Fractal Rendering (Correctness)	10	Correct threaded output, pixel-identical to serial reference on both views
Part 1 – Fractal Rendering (Load Balancing)	10	3-thread anomaly correctly diagnosed; improved mapping reaches roughly 7–8x at 8 threads
Part 2 – SIMD Vectorization	20	Correct <code>clampedAltVector()</code> for any N and <code>VECTOR_WIDTH</code> , including tail handling and masks
Part 3 – AXPBY	20	Correct threaded <code>axpbyThread.cpp</code> ; contiguous decomposition; passes <code>--check</code> for uneven N
Written Analysis	15	Sound reasoning for the 3-thread anomaly, SIMD utilization trend, and compute- vs. memory-bound comparison

Evaluation	25	Can correctly explain own code, own numbers, and own design decisions
Total	100	

8. Tips for Success

- Start early. All three take real debugging time; don't leave them until the last day.
- Always rebuild (**make**) before timing. Stale builds give misleading numbers.
- Use **--check** after every change; a fast, wrong answer is not a correct answer.
- Run **--sweep** multiple times; take the best or median of a few runs, since timing has noise.
- Write your explanations as you go, right after you observe each result. It is much harder to reconstruct your reasoning the night before submission.